

Classifiers & Preprocessing Cheat Sheet

Four Classifiers — Same Interface

```
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier

clf = LogisticRegression(max_iter=1000)    # or
KNeighborsClassifier(n_neighbors=5)
clf.fit(X_train, y_train)                  # SVC(),
DecisionTreeClassifier(max_depth=4)
clf.predict(X_test)
```

Missing Data

```
from sklearn.impute import SimpleImputer
imp = SimpleImputer(strategy="median")
imp.fit_transform(X_train[num_cols])
```

strategy: "mean" | "median" | "most_frequent"
Median is more robust to outliers/spikes.

Categorical Encoding

```
from sklearn.preprocessing import OneHotEncoder
enc = OneHotEncoder(handle_unknown="ignore")
enc.fit_transform(X_train[cat_cols])
```

handle_unknown="ignore" avoids crashing on an
unseen category at test/deploy time.

ColumnTransformer + Pipeline (the Week 2 pattern)

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

numeric_transformer = Pipeline([("imputer",
                                SimpleImputer(strategy="median")),
                                ("scaler",
                                 StandardScaler())])
categorical_transformer =
OneHotEncoder(handle_unknown="ignore")

preprocessor = ColumnTransformer([("num",
                                   numeric_transformer, numeric_features),
                                   ("cat",
                                    categorical_transformer, categorical_features)])

pipe = Pipeline([("preprocess", preprocessor), ("clf",
LogisticRegression())])
pipe.fit(X_train, y_train)    # imputer/scaler/encoder
all fit on TRAIN only
```

Which Classifier?

Trees: no scaling needed, interpretable
Logistic / SVM / k-NN: scale features first
Small, clear-margin data → SVM
Need a fast, interpretable baseline → Logistic

Common Gotchas

A raw string column into a classifier → ValueError.
Fit preprocessing on X_train, never on X_test.
Class imbalance? Check with y.value_counts().
Compare models on the SAME train/test split.

Coming in Week 3: swapping the classifier for a **regressor** — same Pipeline pattern, continuous target instead of a class label.